CPSC 121: Models of Computation

Module 11: Big-O and the Working Computer

CPSC 221/320

CPSC 213/313

Jordon Johnson

Adapted from slides by Karina Mochetti, Steve Wolfman, Seyyed Hosseini, Meghan Allen, Patrice Belleville, …

Learning Goals

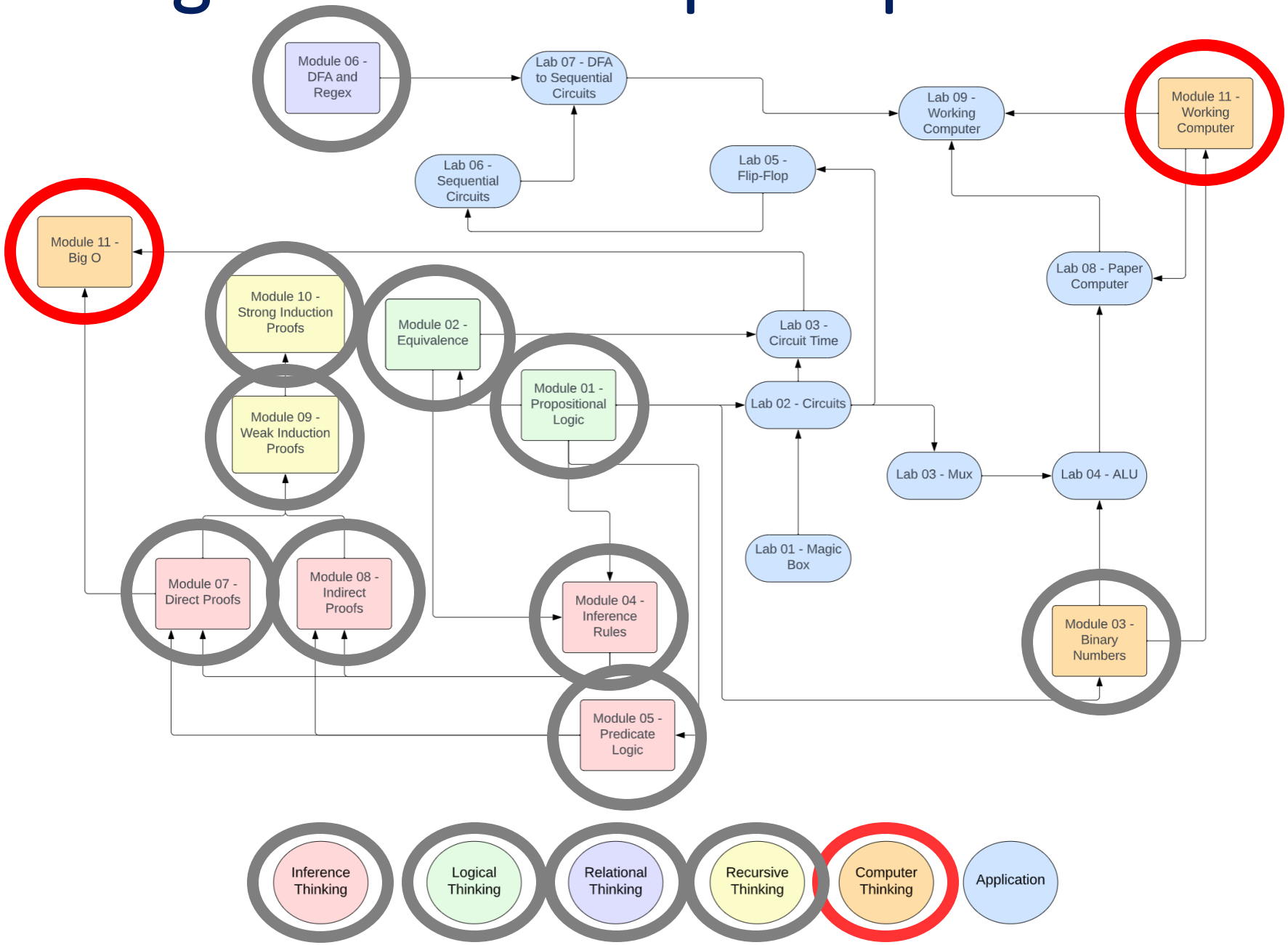
By the end of the unit, you should be able to:

- Define and compare big-O, big-Omega, and big-Theta notations.
- Evaluate if a function f is in big-O of a function g.
- Use direct proofs to show that a function f is in big-O of a function g.
- Comprehend how big-O is used to analyze an algorithm's efficiency.
- Specify the overall architecture of a (Von Neumann) stored program computer — an architecture that instantiates the principle that both program and data are bits (i.e., states) loaded and stored in common memory.
- Trace execution of instructions through a working computer: the basic fetch-decode-execute instruction cycle and the data flow to/from the arithmetic logic unit (ALU), the main memory, and the Program Counter (PC).

Module Outline

- **“We are here”, reading review, quiz notes**
- Algorithm efficiency
- Proving Big-O
- Worksheet
- Our working computer
- Extra stuff

CPSC 121 High-Level Concept Map



CPSC 121: The Big Stories

- **THEORY**
  - How do we model computational systems?
  - **NOW:** Proofs about what is **possible** and **efficient** to compute!

- **HARDWARE**
  - How do we build devices that can compute?
  - **NOW:** Discussion of the **working computer** from the lab!

Reading Review

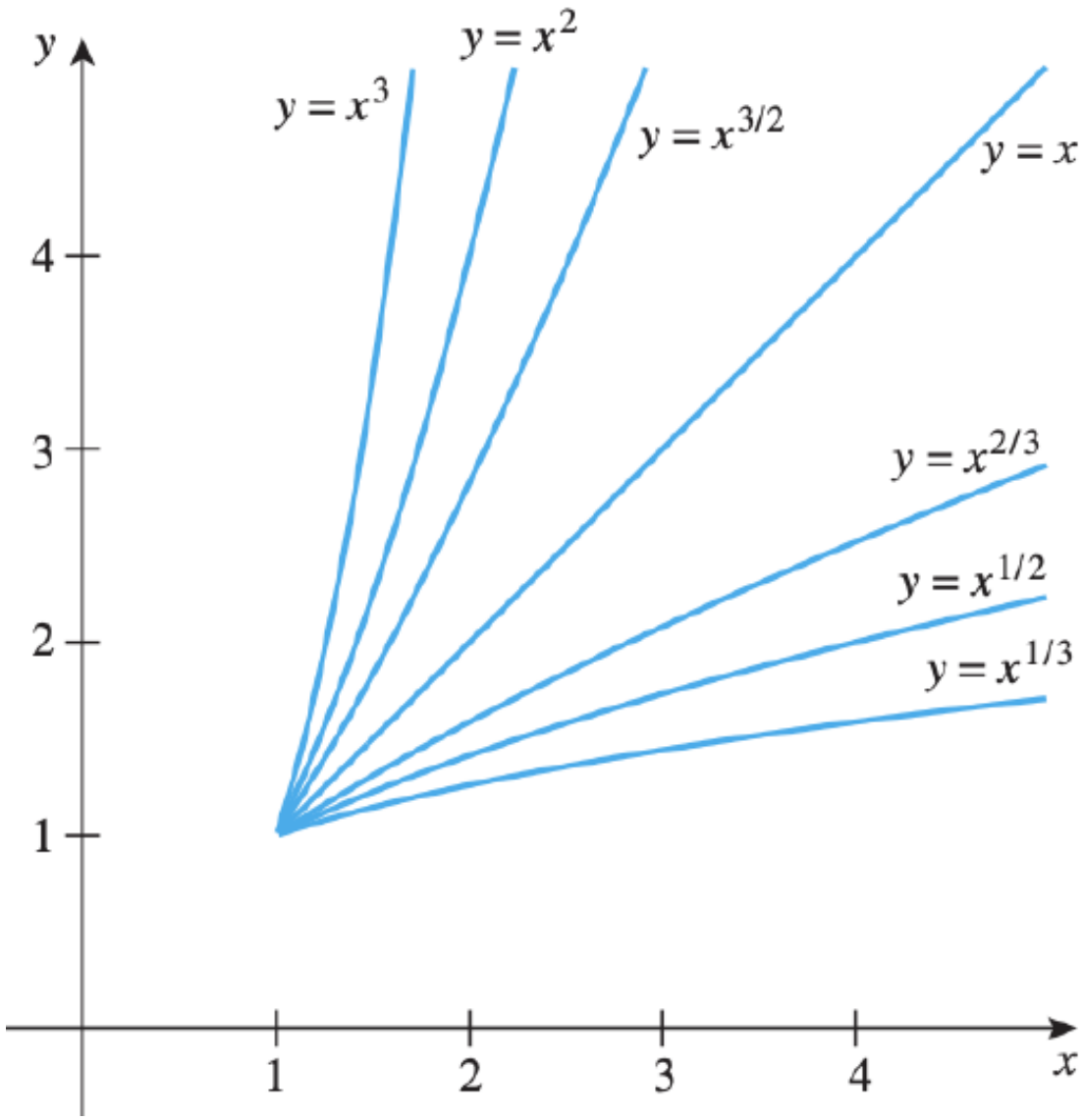
- **Big-O**
  - f is of order **at most** g, written $f(n) \in O(g(n))$, **iff** there exists a positive real number c and a natural number n_0 such that $f(n) \leq c * g(n)$ for all natural numbers $n \geq n_0$.
- **Big-Ω**
  - f is of order **at least** g, written $f(n) \in \Omega(g(n))$, **iff** there exists a positive real number c and a natural number n_0 such that $c * g(n) \leq f(n)$ for all natural numbers $n \geq n_0$.
- **Big-θ**
  - f is of order g, written $f(n) \in \Theta(g(n))$, **iff** there exists positive real numbers c_1, c_2 and a natural number n_0 such that $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all natural numbers $n \geq n_0$.

Reading Review

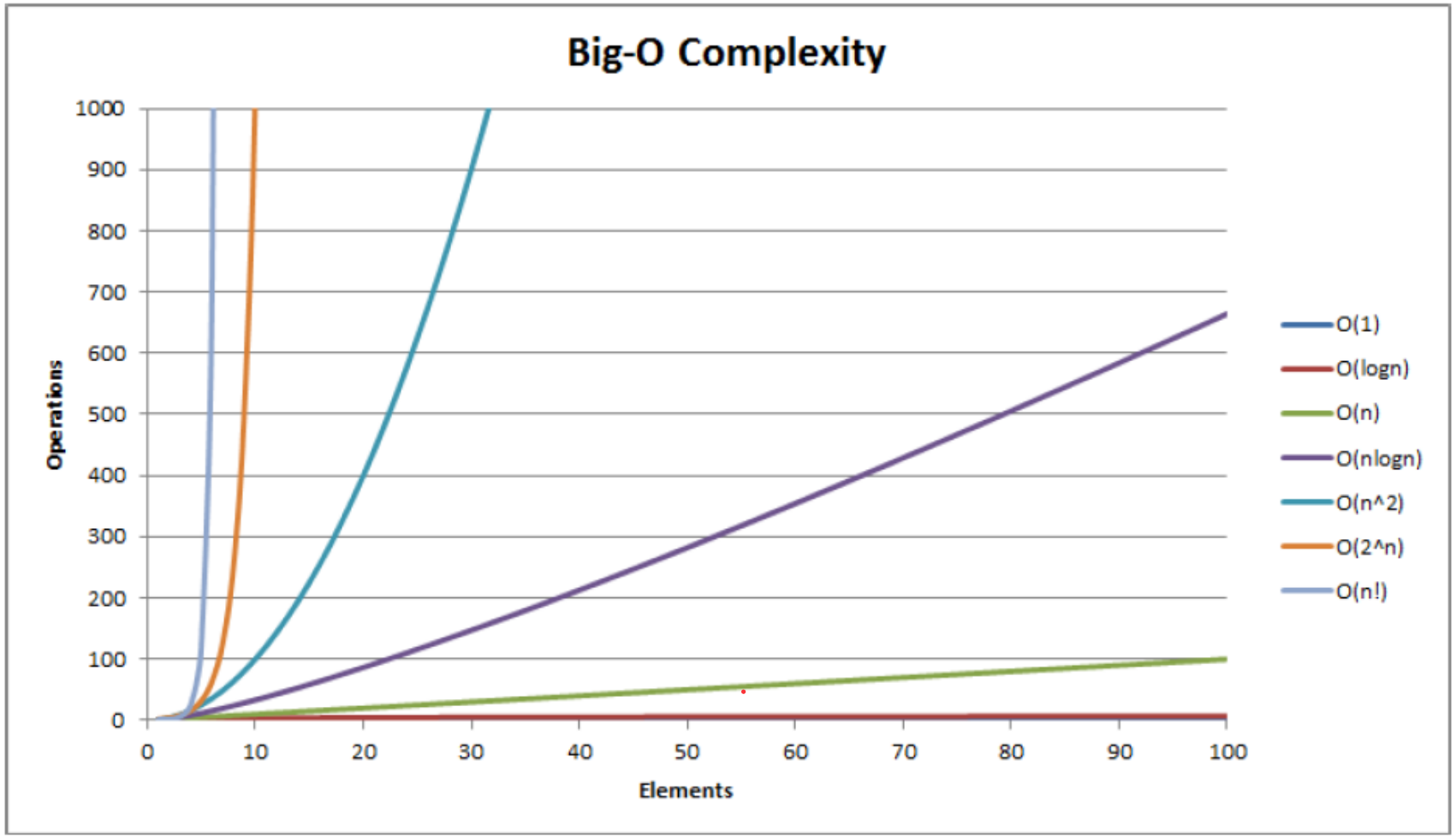
- **Properties:**
  - $\left(f(x) \in O\left(g(x)\right)\right) \wedge \left(f(x) \in \Omega\left(g(x)\right)\right) \leftrightarrow \left(f(x) \in \Theta\left(g(x)\right)\right)$
  - $\left(f(x) \in \mathrm{O}\left(g(x)\right)\right) \leftrightarrow \left(g(x) \in \Omega\left(f(x)\right)\right)$
  - $\left(f(x) \in \Theta\left(g(x)\right)\right) \leftrightarrow (g(x) \in \Theta\left(f(x)\right))$

Reading Review

- **Orders of Power Functions**
  - $(r < s) \rightarrow (x^r \in O(x^s))$



Reading Review



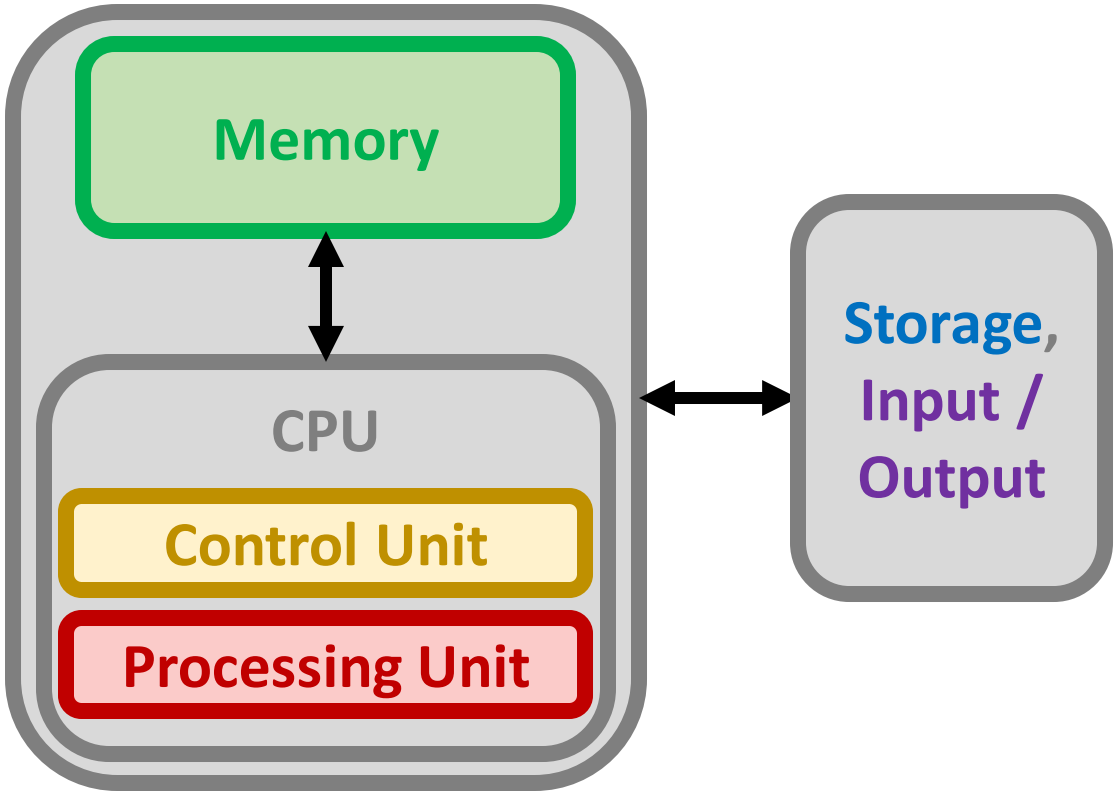
$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

tractable | not tractable

Reading Review

The **Von Neumann Architecture** consists of:

- A **processing unit** that contains an arithmetic logic unit (ALU) and processor **registers**
- A **control unit** that contains an instruction register and **program counter**
- **Memory** that stores data and instructions
- External mass **storage**
- **Input and output** mechanisms



Reading Review

Machine code

- **Moves:** move data to/from registers
  - e.g. **irmovl 0x42, %edx**
- **Operations:** calculate arithmetic and logic operations between constants and/or values in registers
  - e.g. **addl %eax, %ebx**
- **Jumps/branches:** modify the order of execution, enabling loops and conditions
  - e.g. **jge label**

Execution stages

- **Fetch/Decode:** read instruction and decide new PC value
- **Execute:** read values from registers and use the ALU to perform computations
- **Memory:** read data from (or write data to) memory
- **Write-back:** store value(s) into register(s)
- **PC update:** store the new PC value

Quiz Feedback

Which statement is correct?

A. $2 \in O(2^x)$
B. $x \in O(2^x)$
C. $x * \log_2 x \in O(2^x)$
D. $x^{10} \in O(2^x)$
E. All of the above
F. None of the above

Where does the Von Neumann architecture store instructions and data?

A. In separate memory spaces
B. On the monitor
C. In the keyboard
D. In the same unified memory space (though perhaps different areas of that space)
E. None of the above

RAM

Module Outline

- “We are here”, reading review, quiz notes
- **Algorithm efficiency**
- Proving Big-O
- Worksheet
- Our working computer
- Extra stuff

Algorithm Efficiency

Alice and Bob are taking CPSC 110. They have different ways to solve a particular problem. Alice is using Algorithm A, and Bob is using Algorithm B.

Each one wants to prove to you that their algorithm is the best one!

iClicker 11.00

Which algorithm is more efficient?

Algorithm A
10 seconds

Algorithm B
35 seconds

A. Algorithm A
B. Algorithm B
C. We consider them equally efficient
D. Not enough information to tell
E. Inconceivable!

Algorithm Efficiency

We want to measure how efficient each algorithm is, in a way that does not depend on:

- the programming language used
- the quality of the compiler/interpreter used
- the speed of the computer used

One idea is to count the number of **elementary steps** of the algorithm.

iClicker 11.01

Which algorithm is more efficient?

Algorithm A
750 steps using the same language, compiler, and processor

Algorithm B
200,000 steps using the same language, compiler, and processor

A. Algorithm A
B. Algorithm B
C. We consider them equally efficient
D. Not enough information to tell
E. You keep using that word. I do not think it means what you think it means.

Algorithm Efficiency

We want to measure how efficient each algorithm is, in a way that also considers the **size** of the data it is using.

- Instead of considering all cases, we use the variable n to give the size of the input
- A step is anything that can be computed in constant time, that is, independent from n.
  - What this means can be **arbitrary** and makes any **constants/coefficients** we use **"unreliable"** to some extent
- The number of steps will thus depend on n.

iClicker 11.02

Which algorithm is more efficient?

Algorithm A

75n steps using the same language, compiler, and processor

Algorithm B

200n steps using the same language, compiler, and processor

A. Algorithm A
B. Algorithm B
C. We consider them equally efficient
D. Not enough information to tell
E. Have fun storming the castle!

Coefficients and Algorithm Efficiency

	n=1	n=10	n=100	n=1,000	n=10,000
3n	3	30	300	3,000	30,000
6n	6	60	600	6,000	60,000
n^2	1	100	10,000	1,000,000	100,000,000

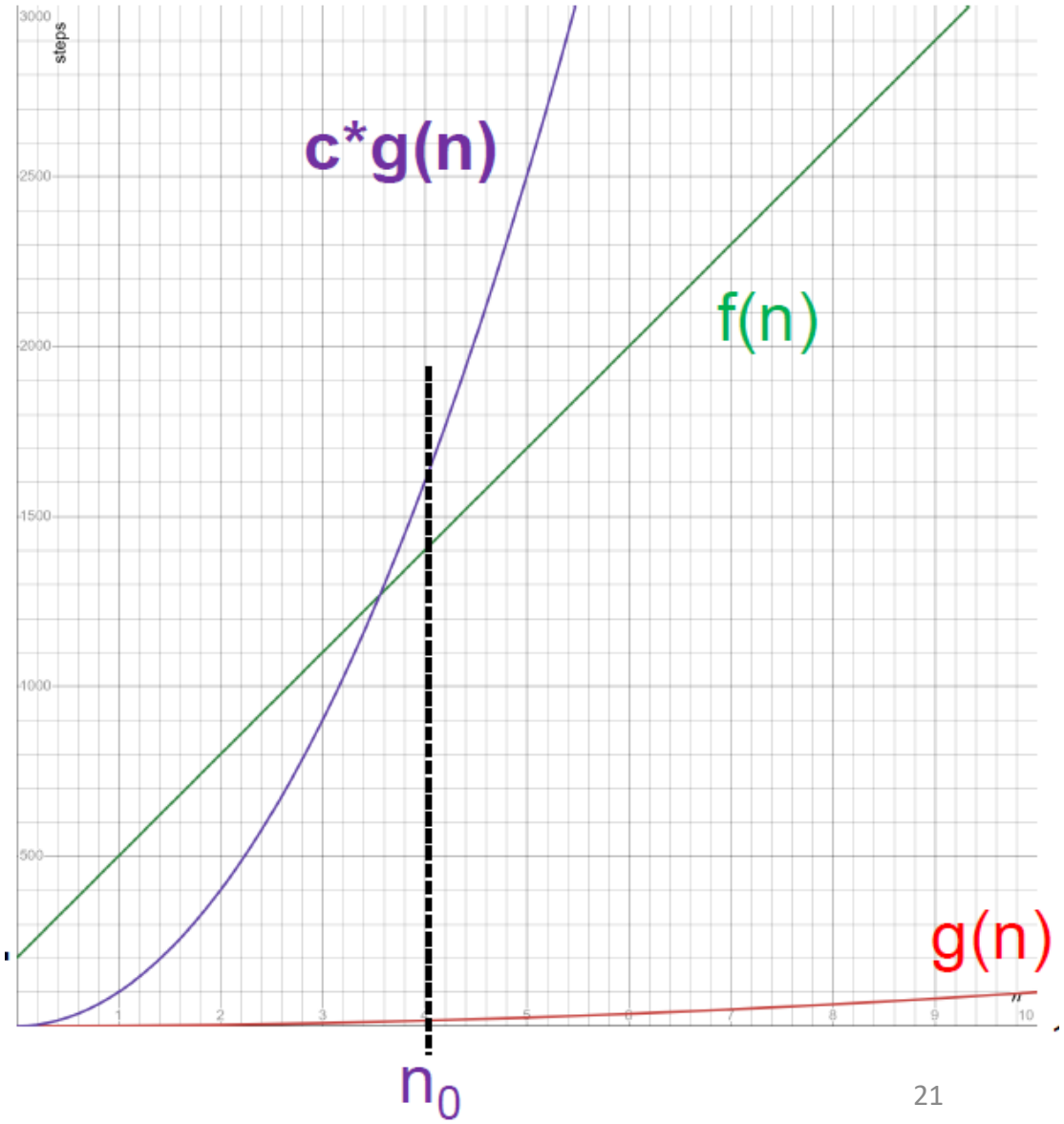
At large scales, differences introduced by potentially unreliable coefficients are quite small compared to differences introduced by different “categories” of algorithms.

Comparing Algorithms with c and n_0

We pick a "speed factor" $c \in \mathbb{R}^+$ that we can use to scale an algorithm up or down, so computer speed or longer/shorter steps don't matter.

We pick a "scaling threshold" $n_0 \in \mathbb{N}$ that we can use to ignore behavior on "small" problem sizes.

- *Fun aside: so long as the functions' values must be positive, n_0 is handy but unnecessary; c is enough.*



Example Problem: Search

Given a sorted list of names with corresponding phone numbers, find the phone number for a given name N.

Algorithm L

- Check the first name. If it's not N, then check the second name. Then the third name, etc. until N is found; return N. If none of the names are n, return NULL.
- Assume this algorithm has approximately **n** steps.

Algorithm B

- Check the name in the middle of the list. If N is earlier alphabetically, search the first half of the list using B; if N is later, search the second half of the list using B. Repeat until N or an empty sublist is found; return N in the former case and NULL in the latter.
- Assume this algorithm has approximately 100* $\log_2 n$ steps.

Example Problem: Search

Algorithm L - Assume this algorithm has approximately **n** steps.
Algorithm B - Assume this algorithm has approximately 100 * $\log_2 n$ steps.

For a list with n = 15 names:

- **L** takes **15** steps
- **B** takes 100 * $\log_2 15$ ≈ **400** steps

For a list with n = 63 names:

- L takes **63** steps
- B takes 100 * $\log_2 63$ ≈ **600** steps

For a list with n = 1,048,575 names:

- L takes **1,048,575** steps
- B takes 100 * $\log_2 1{,}048{,}575$ ≈ **2000** steps

Time of Execution

How long would an algorithm with f(n) operations take to execute on a computer that performs **10^{12} operations per second**?

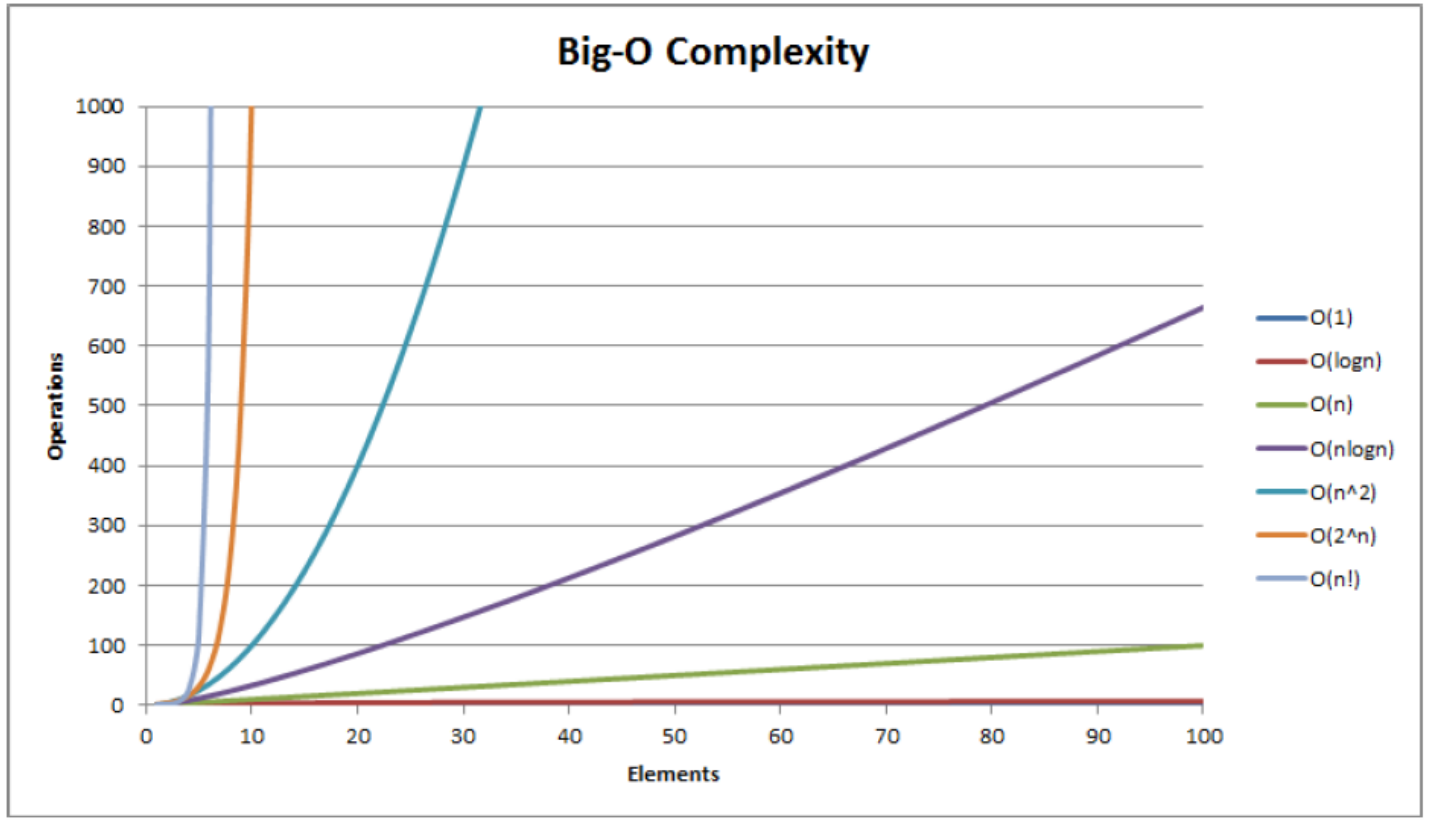
f(n)	n = 10	n = 100	n = 100	n = 10,000	n = 10^{12}
n	10^{-11}s	10^{-10}s	10^{-9}s	10^{-8}s	1s
n log n	10^{-11}s	10^{-9}s	10^{-8}s	10^{-7}s	40s
n^2	10^{-10}s	10^{-8}s	10^{-6}s	10^{-4}s	10^{12}s
n^3	10^{-9}s	10^{-6}s	10^{-3}s	1s	10^{24}s
2^n	10^{-9}s	10^{18}s	10^{289}s	too long	too long

10^4s ≈ 3 hours
10^{12}s ≈ 32 years
10^{18}s ≈ 30 billion years

10^{80} ≈ the estimated number of particles in the observable universe

Recap: Big-O

O(g(n)) is a **set of functions** that **grows at most** at g(n).



$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

Big-O: Approximations

Assuming that f(n) is the function describing your function's performance, we need to find which O(g(n)) it fits.

Eliminate low order terms

For a sum of terms, we can eliminate all but the highest-order term.

Eliminate constants/coefficients

For a given term, we can eliminate constant coefficients. If the term is only a constant, we can reduce it to 1.

- $4n + \cancel{5}$ $\Rightarrow \cancel{4}n$ $\Rightarrow n$ $\in O(n)$
- $0.5n \log_3 n - \cancel{2n + 7}$ $\Rightarrow \cancel{0.5}n \log_3 n$ $\Rightarrow n \log n$ $O(n \log n)$
- $2^n + \cancel{n^3} + \cancel{3n}$ $\Rightarrow 2^n$ $\Rightarrow 2^n$ $O(2^n)$
- $5n^3 + \cancel{3n^2} + \cancel{6n}$ $\Rightarrow \cancel{5}n^3$ $\Rightarrow n^3$ $O(n^3)$
- $\cancel{10^{42}n} + \cancel{10^{42} \log n} + 10n^4$ $\Rightarrow \cancel{10}n^4$ $\Rightarrow n^4$ $O(n^4)$

iClicker 11.03

Consider the statements below:

- $n \in O(n)$
- $n \in O(n \log n)$
- $n \in O(n^2)$
- $n \in O(n^3)$
- $n \in O(2^n)$
- $n \in O(n!)$

How many of those statements are correct?

A. 1
B. at most 3
C. at most 5
D. all of them
E. none of them

Big-O: Approximations

O(g(n)) is a **set of functions** that **grows at most** at g(n).

In practice, we tend to want to find the smallest applicable g(n) since we are trying to show that our algorithm is efficient (i.e. that its f(n) does not grow fast).

- *More on this in CPSC 221* ☺

In our search example:

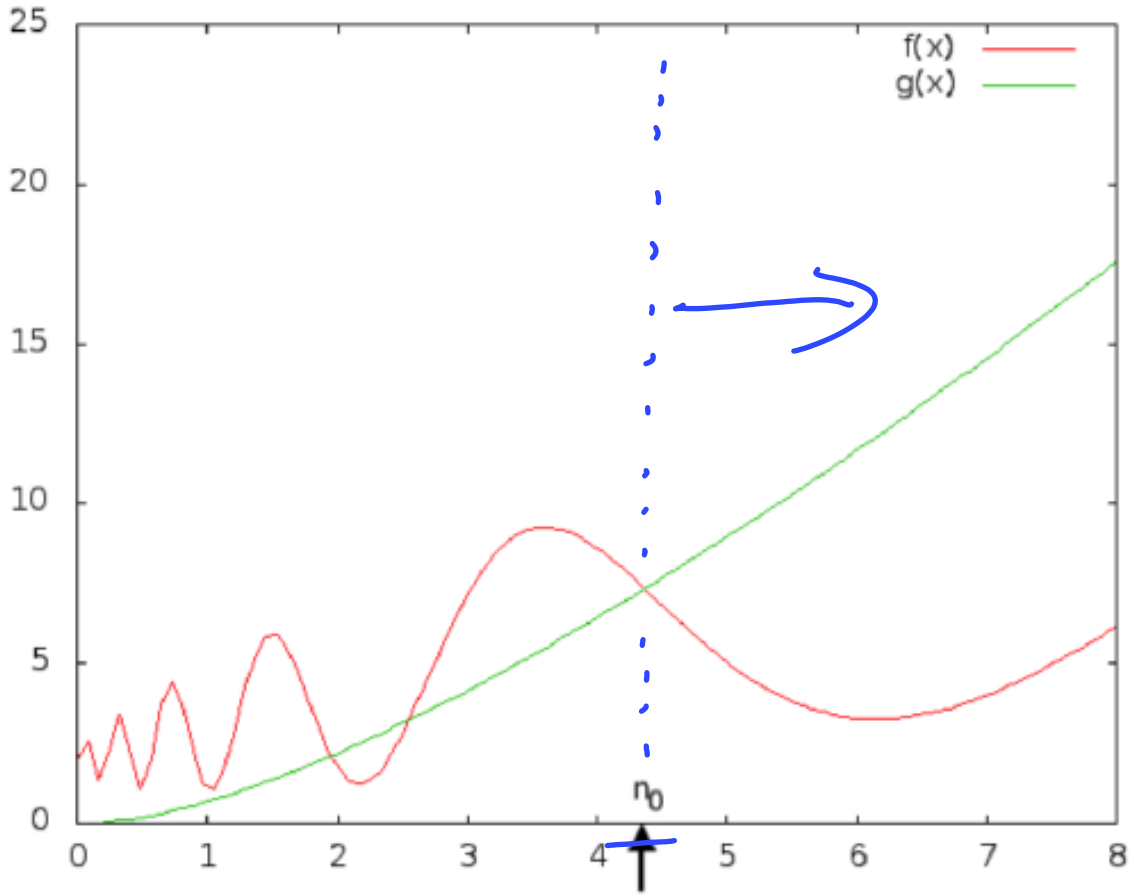
- Algorithm L (i.e. **linear search**) runs in O(n) time.
- Algorithm B (i.e. **binary search**) runs in O(log n) time.

So, we consider Algorithm B to be "better"!

Big-O: Definition

f(n) is in O(g(n)):

$$\exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}, \forall n \in \mathbb{N}, n \geq n_0 \rightarrow f(n) \leq c * g(n)$$



- f(n) grows **no faster** than g(n)
- g(n) is an **upper bound** on the **growth rate** of f(n)

Module Outline

- “We are here”, reading review, quiz notes
- Algorithm efficiency
- **Proving Big-O**
- Worksheet
- Our working computer
- Extra stuff

Proving Big-O

Rules for inequalities:

- When dealing with **<**, you may make the right-hand-side expression **larger**, but never **smaller**.

- When dealing with **>**, you may make the right-hand-side expression **smaller**, but never **larger**.

Proving Big-O: Example

$$\exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}, \forall n \in \mathbb{N}, n \geq n_0 \rightarrow f(n) \leq c * g(n)$$

$$f(n) = 8n^2 - 14n \qquad g(n) = 2n^3$$

Proof:

Choose c = 4 and n_0 = 0

Consider an unspecified natural number $n \geq n_0$.

$$\begin{aligned} f(n) &= 8n^2 - 14n \\ &\leq 8n^2 && \text{for } n \geq 0 \\ &\leq 8n^3 && \text{for } n \geq 0 \\ &\leq (4 * 2)n^3 \\ &\leq 4 * 2n^3 \end{aligned}$$

Therefore, $f(n) \leq c * g(n)$ for $n \geq n_0$.

QED

Proving Big-O: Example

$$\exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}, \forall n \in \mathbb{N}, n \geq n_0 \rightarrow f(n) \leq c * g(n)$$

$$f(n) = 4n + 20 \qquad g(n) = 2n^2 - 10n$$

Proof:

Choose c = 1 and n_0 = 17

Consider an unspecified natural number $n \geq n_0$.

$$\begin{aligned}
f(n) &= 4n + 20 \\
&= 4n + 20 + 10n - 10n \\
&\leq 4n + 20n + 10n - 10n && \text{for } n \geq 1 \\
&\leq 34n - 10n \\
&\leq 17 * (2n) - 10n \\
&\leq 17 * (2n) * \frac{n}{17} - 10n && \text{for } n \geq 17 \\
&\leq 2n^2 - 10n
\end{aligned}$$

Therefore, $f(n) \leq c * g(n)$ for $n \geq n_0$.

QED

Proving Big-O: Step by Step

- Start your proof by leaving the values of c and n_0 blank.
- Write out f(n) = <*whatever f(n) is*>.
  - **DO NOT ASSUME** that it is ≤ c * g(n).
- Change the right side of the equation to reach c * g(n).
- Go back to the start of your proof and fill in the values of c and n_0.
  - Note that n_0 will be the largest value used for limiting n in the justified steps.

- Each algebraic step should be small and properly justified unless really obvious.
  - Make sure to check if each step is true for all stated values of n.

Module Outline

- “We are here”, reading review, quiz notes
- Algorithm efficiency
- Proving Big-O
- **Worksheet**
- Our working computer
- Extra stuff

Worksheet 11.01

Prove that $\underbrace{n^2 + 3n + 5}_{f(n)} \in O(\underbrace{n^2}_{g(n)})$.

Proof:

Let $c = \underline{9}$ and $n_0 = \underline{1}$

$f(n) = n^2 + 3n + 5$

$\leq n^2 + 3n^2 + 5n^2$ for $n \geq 1$

$= 9n^2$

Thus there exist $c \in \mathbb{R}^+$ and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$

$\blacksquare$

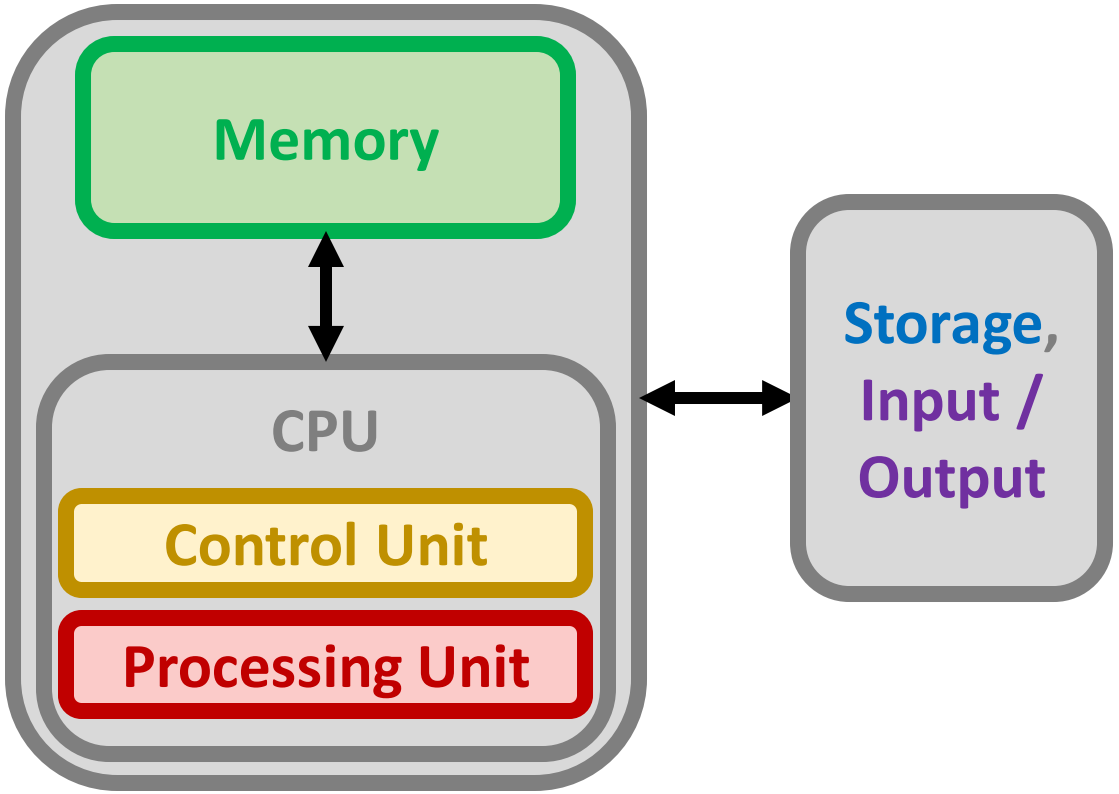
Module Outline

- “We are here”, reading review, quiz notes
- Algorithm efficiency
- Proving Big-O
- Worksheet
- **Our working computer**
- Extra stuff

Recap: the Von-Neumann Architecture

The **Von Neumann Architecture** consists of:

- A **processing unit** that contains an arithmetic logic unit (ALU) and processor **registers**
- A **control unit** that contains an instruction register and **program counter**
- **Memory** that stores data and instructions
- External mass **storage**
- **Input and output** mechanisms

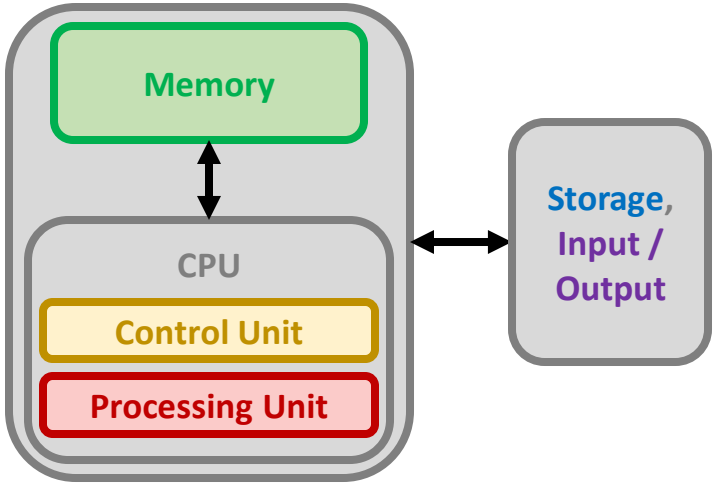


A Working Computer: Von Neumann Architecture

address of the 4-byte value

address	…	0x00000420	0x00000421	0x00000422	0x00000423	…
value	…	00000001	00010011	10011010	10011110	…

18,061,982



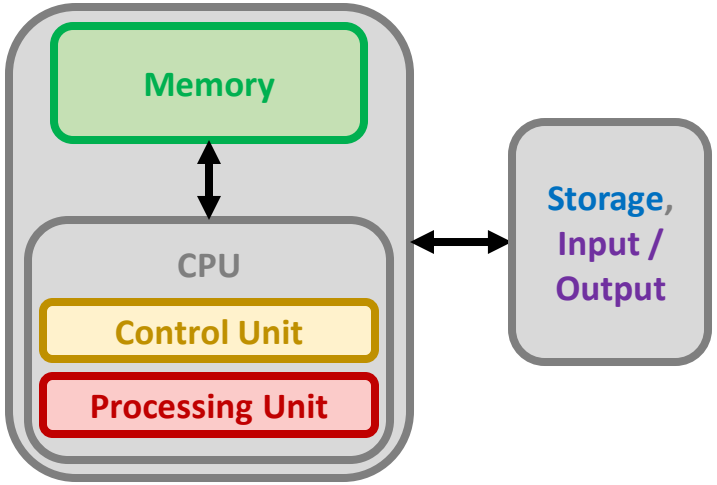
Memory:

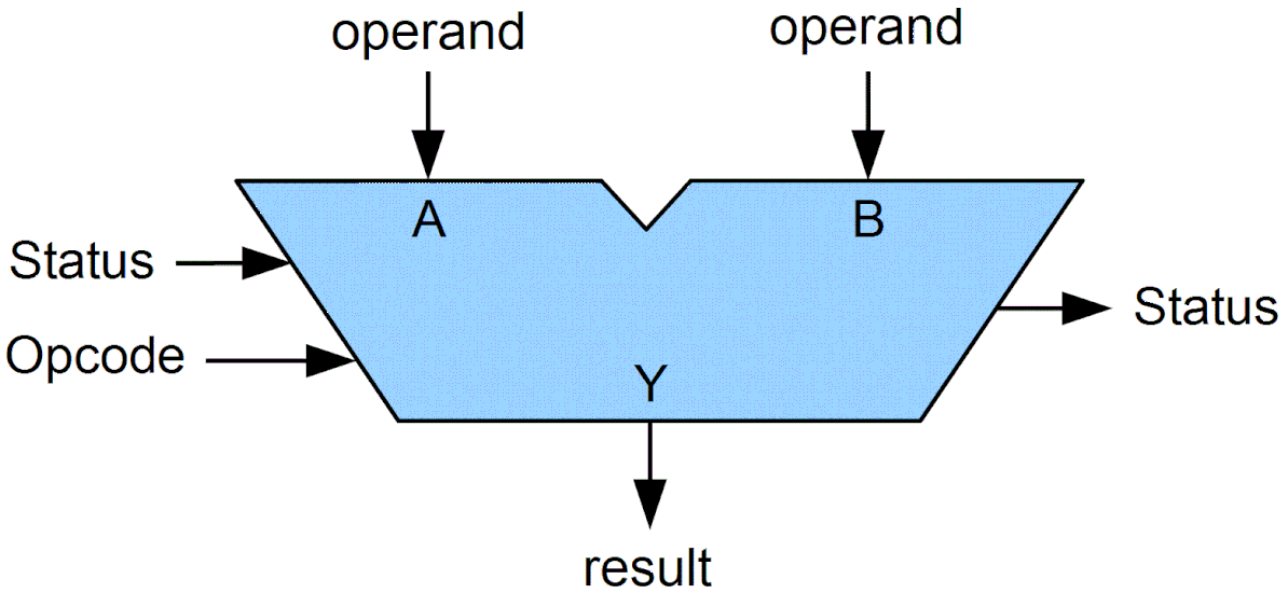
- Contains instructions and data *(same binary representation)*
- Divided into memory locations, like a list or array
  - Just as each array element has an **index**, each memory location has an **address**
  - 32-bit computers can have addresses with 32 bits (2^{32} addresses, or 4GB); 64-bit computers can have 2^{64} addresses
- Each location has a fixed number of bits (usually 8 bits = 1 byte)
- Values using more than 8 bits are stored in multiple consecutive memory locations
  - the address of the value is the address of the **first byte** of the value (i.e. the **lowest** memory address being used)

A Working Computer: Von Neumann Architecture

Arithmetic Logic Unit (ALU):

- Performs arithmetic and logical operations.
- Can detect overflow and other status.
- Opcode identifies the operation being performed
  - +, -, *, /, AND, OR, …

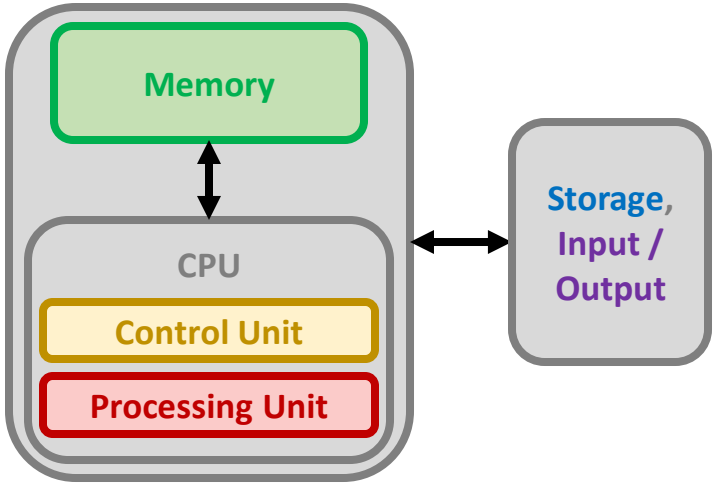




A Working Computer: Von Neumann Architecture

Control Unit:

- Decides which instructions to execute.
- Executes these instructions in a sequence.
  - *Not quite true, but this is how it appears to the user.*
- Contains the **Program Counter (PC)**, a special register that has the memory address of the next instruction.



PC →

```
irmovl 0x3, %eax
irmovl 0x35, %ebx
subl %eax, %ebx
halt
```

current instruction

Our Working Computer

- Implements the design presented in the textbook by Bryant and O'Hallaron (CPSC 213/313)
- A small subset of the IA32 (Intel 32-bit) architecture

It has:

- 12 types of instructions
- 1 program counter register (PC)
- 8 general-purpose 32-bit registers, used for values with which we are currently working
  - Registers are faster to access then memory, and so values are often copied to/from memory but are "used" in registers.

Examples of Instructions

- `irmovl 0x1A, %ecx`
  - loads the constant value 0x1A (26 in decimal) into register `ecx`.
- `subl %eax, %ebx`
  - subtracts register `eax` from register `ebx` and loads the result in `ebx`.
- `jge label`
  - called a *conditional jump instruction*, it changes the value of the program counter according to the result of the most recent ALU operation.

iClicker 11.04

What does this sample program do?

```
irmovl 0xE9, %eax
irmovl 0x0C, %ebx
addl %ebx, %eax
halt
```

A. Adds values and loses the result
B. Moves values to two registers and nothing else
C. Adds values and saves the result to a register
D. Generates an error
E. Divides by zero and destroys the universe MWAHAHAHAHA

Sample Program

```
foo: irmovl 0x35, %eax
     irmovl 0x3, %ebx
     subl %eax, %ebx
     jge foo
     halt
```

if ALU result had been ≥0

loads 0x35 (53) into register eax

loads 0x3 (3) into register ebx

subtracts eax from ebx and loads the result into ebx: 3 – 53 = -50

checks if the last ALU result was >= 0

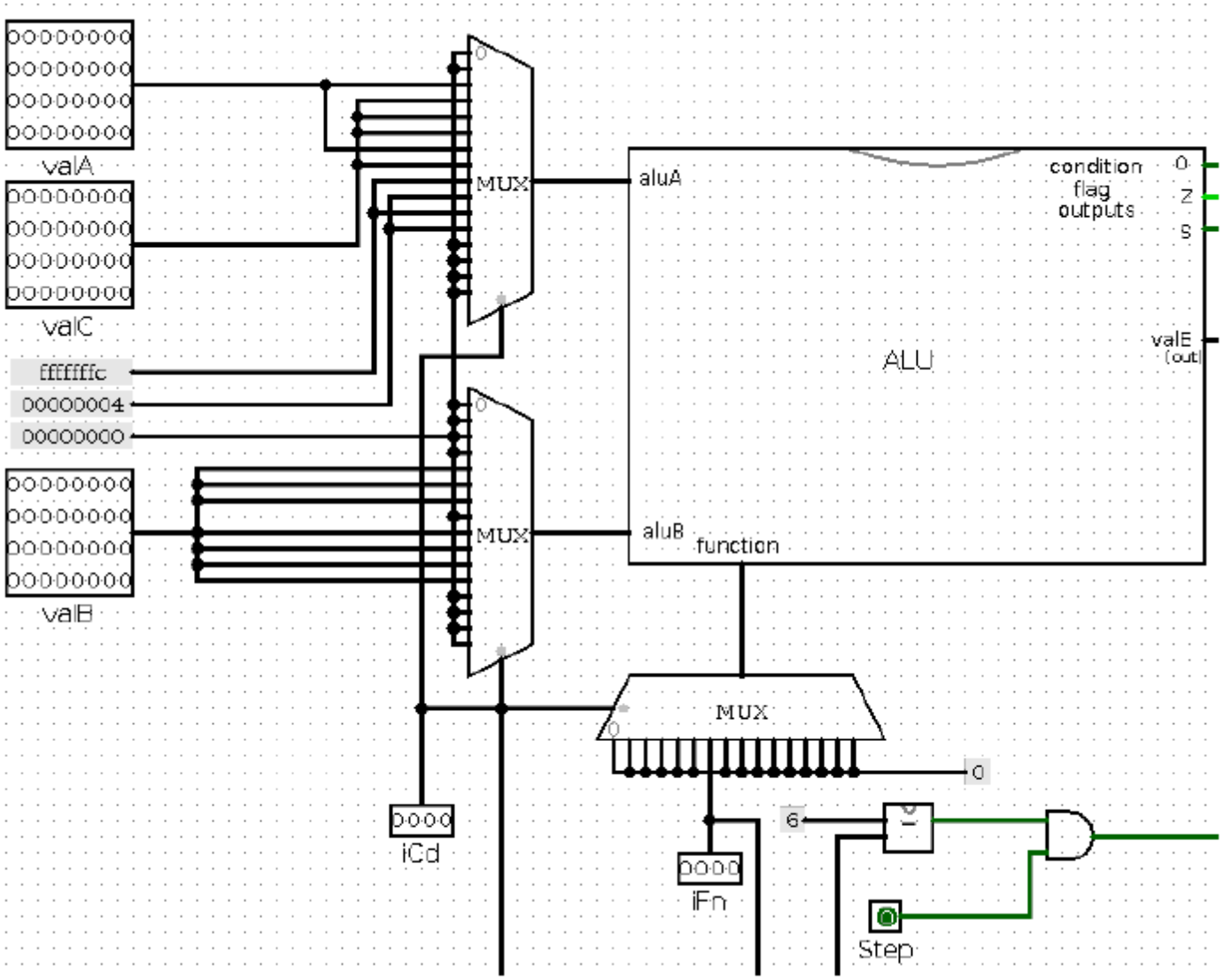
- if it had been, would have set PC to the instruction with the label `foo`
- since it wasn't in this case, the program will proceed to the next instruction

ends the program (halts)

A Working Computer

How does the computer know which instruction does what?

- Each instruction is a sequence of bits
- Some of the bits identify which instruction it is
- Other bits specify the operands to use
- These bits are used as select inputs for several **multiplexers**



Instructions

subl %eax, %ebx	irmovl 0xCAFE, %ecx	jge foo
0110 0001 0000 0011 6103	30F10000CAFE	7500004200
arithmetic / logic operation	load constant into register	conditional jump
subtraction	not used	greater than or equal
eax	no register used	
ebx	ecx	
	constant value 0x0000CAFE	address of label ‘foo’

Sample Program

```
foo: irmovl 0x35, %eax
     irmovl 0x3, %ebx
     subl %eax, %ebx
     jge label
     halt
```

0x189	0x18A	0x18B	0x18C	0x18D	0x18E	0x18F	0x190	0x191	0x192	0x193	0x194	0x195	0x196	0x197	0x198	0x199	0x19A	0x19B	0x19C
30	F0	00	00	00	35	30	F3	00	00	00	03	61	03	75	00	00	01	89	00

- The program starts at PC = 0x189
  - The value of the label ‘foo’ is 0x189
- The value of PC updates from 0x189 (mov) to 0x18F (mov) to 0x195 (sub) to 0x197 (jge) to either 0x189 (foo) or 0x19C (halt)
- The program ends on the instruction 00 (halt)

Sample Program

This CPU divides instruction execution into 6 stages:

- **Fetch**/**Decode**: read instruction and decide on a new PC value
- **Execute**: read values from registers and use the ALU to perform computations
- **Memory**: read data from or write data to memory
- **Write-back**: load value(s) into register(s)
- **PC update**: store the new PC value

Notes:

- Not all stages do something for every instruction.
- The number of stages may vary depending on the CPU architecture *(as you will see in CPSC 213/313).*

Executing an Instruction

```
irmovl 0xCAFE, %ecx
```

1. **Fetch**/**Decode**:
   - instruction ← 30F10000CAFE
   - nextPC ← currentPC + 6
3. **Execute**:
   - valC ← CAFE
   - valE ← valC

   Why all of this? CPSC 313
4. **Memory**:
   - nothing needs to be done
5. **Write-back**:
   - %ecx ← valE
6. **PC update**:
   - PC ← nextPC

Executing an Instruction

```
subl %eax, %ebx
```

1. **Fetch**/**Decode**:
   - instruction ← 6103
   - nextPC ← currentPC + 2
3. **Execute**:
   - valA ← %eax
   - valB ← %ebx
   - valE ← valB - valA
4. **Memory**:
   - nothing needs to be done
5. **Write-back**:
   - %ebx ← valE
6. **PC update**:
   - PC ← nextPC

Executing an Instruction

```
jge foo
```

1. **Fetch**/**Decode**:
   - instruction ← 750000CAFE
   - nextPC ← currentPC + 5
3. **Execute**:
   - valC ← CAFE
   - bch ← 1 or 0 (depending on valE)

was branch condition met or not

4. **Memory**:
   - nothing needs to be done
5. **Write-back**:
   - nothing needs to be done
6. **PC update**:
   - if bch is 0 then PC ← nextPC; if 1 then PC ← valC

iClicker 11.05

What can we say about the program counter (PC)?

A. It is incremented by 4 after each instruction
B. It is only updated after some instructions
C. It has the memory address of the next instruction to be executed
D. It is updated during the Memory stage of execution
E. None of the above

Coding with Instructions

The greater irony was that RollerCoaster Tycoon otherwise stood as a monument to what a single person can accomplish in programming. Written almost entirely in assembly code (like Sawyer's previous Transport Tycoon), RollerCoaster Tycoon and its sequel squeezed and re-squeezed the processors of the time to simulate rides, economies, and up to thousands of visitors and their states of mind. Churning through so many numbers in real-time without hitching demanded a lean, uncompromising approach and not the slower, more user-friendly C family of languages. And in ultra-lean assembly, where letters stand in for ones and zeroes, one speaks directly to the processor.

Read more:
https://pcgamesn.com/rollercoaster-tycoon/code-chris-sawyer

Models of Computation

We have discussed several models of computation and what they **can** do…

- Combinatorial circuits
- Sequential circuits
- DFAs
- ~~Turing Machines~~

But what can a computer **not** do?

Example: **The Halting Problem**

- Given a program P and an input I, will P halt if we run it using input I?
- **Theorem:** It is **not possible** to write a program that solves the halting problem.

Proof: https://www.youtube.com/watch?v=92WHN-pAFCs

Module Outline

- “We are here”, reading review, quiz notes
- Algorithm efficiency
- Proving Big-O
- Worksheet
- Our working computer
- **Extra stuff**

What's Next for CS Theory?

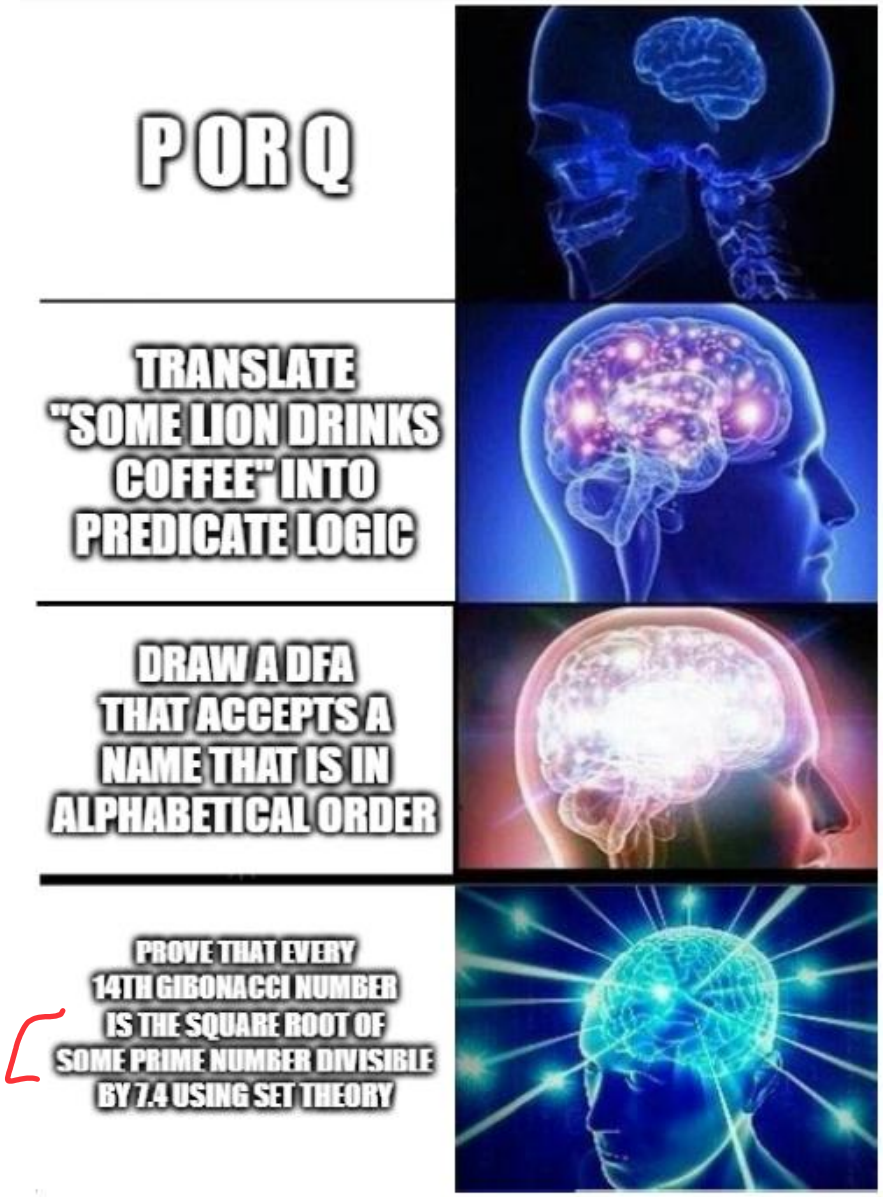
- **CPSC 221:** How do we organize data to solve common computational problems quickly and efficiently? Binary trees that balance themselves! "Arrays" with constant time lookup but strings as indices! Magic!
- CPSC 320: How should we approach algorithmic problems in general? What are high-level approaches that will serve us well, and how can we employ them? Intern/co-op matching! Clustering! Memoization magic (yes, memorization with no "r").
- CPSC 420: What problem do we solve next?!!
- CPSC 421: How do we model computing next?!!
- CPSC 436R: Algorithms are better with random numbers!
- CPSC 436Q: Algorithms are better with quantum entanglement!
- CPSC 302/303/406: How to (not-too-in)correctly solve big problems like physical simulation?
- CPSC 311: How to use theory to design usable programming languages?
- CPSC 322/340/422/440: Can we design algorithms that "learn" and/or "think"?
- …

What's Next for CS Systems/Hardware/etc.?

- **CPSC 213:** What actually happens when you run your code? How can we write code that will behave as expected even if we don't know when events will occur?
- CPSC 313: How can we make CPUs more efficient? How can we make memory accesses more efficient? How can we use this information to make our code more efficient? How should we organize information in storage devices? (And much more…)
- CPSC 314: What do graphics cards do, and how do they work?
- CPSC 317: How do networks…work?
- CPSC 411: How do we write software that will translate code into a format that a CPU can read?
- CPSC 415: How do operating systems…operate?
- CPSC 416/418: How can we "spread out" our computation?
- …

Deep Thought

The CPSC 121 process:



prime numbers don't have integer square roots :(